

DevOps Interview Preparation Guide

Introduction & Experience • CI/CD with Jenkins • Scenario-Based Troubleshooting

01 SECTION 1: Introduction & Experience

Q1. Tell me about yourself and your DevOps journey.

■ Background & Pivot:

I hold a B.Tech in Aeronautical Engineering but pivoted to DevOps after recognizing the industry demand and my strong interest in automation, cloud infrastructure, and systems reliability. I pursued a structured DevOps learning track covering Linux, Docker, AWS, Kubernetes, and CI/CD.

■ Internship at Hisan Labs:

During my 9-month internship at Hisan Labs, Pune, I worked on real cloud infrastructure — provisioning AWS environments with Terraform, building Jenkins CI/CD pipelines, containerizing microservices with Docker and Kubernetes on EKS, and setting up monitoring using Datadog and Grafana.

■ Flagship Project:

My primary project is ApnaKart — a 10-microservice cloud-native e-commerce platform on AWS EKS. It includes Terraform modules, Jenkins pipelines with branch-based environments, Kustomize overlays, IRSA/OIDC, HPA, and Datadog APM. This project captures most of my practical experience end-to-end.

■ Where I'm Headed:

I want to deepen my expertise in platform engineering, GitOps workflows, and SRE practices. I'm actively preparing for AWS Solutions Architect certification to complement my hands-on skills.

Q2. What are your current roles and responsibilities at your current company?

■ Infrastructure Provisioning:

Designed and managed Terraform modules to provision AWS VPC, EKS clusters, RDS (PostgreSQL), S3, CloudFront, and Route53. Maintained remote state in S3 with DynamoDB locking.

■ CI/CD Pipeline Ownership:

Built and maintained Jenkins declarative pipelines with stages for code checkout, Maven build, SonarQube quality gate, Docker image build, Trivy vulnerability scan, ECR push, and Kustomize-based Kubernetes deployment.

■ Container Orchestration:

Managed Kubernetes workloads on EKS — writing Deployments, Services, ConfigMaps, HPA configs, and Kustomize overlays for dev/staging/prod. Configured IRSA for pod-level AWS IAM access.

■ Monitoring & Alerting:

Integrated Datadog APM for distributed tracing, set up Prometheus + Grafana dashboards for infrastructure metrics, and configured alerting thresholds for CPU, memory, and error rates.

■ Security & Compliance:

Ran Trivy image scans in pipelines, managed IAM roles with least-privilege policies, and enforced Kubernetes RBAC with namespace-scoped roles and role bindings.

Q3. Describe the architecture of your current project.

■ Overview:

ApnaKart is a cloud-native e-commerce platform consisting of 10 microservices — including API Gateway, Auth Service, Product Catalog, Cart, Order, Notification, and Payment — all deployed on AWS EKS in a custom VPC.

■ Networking Layer:

Custom VPC with public and private subnets across two AZs. Public subnets host the ALB and NAT Gateway; private subnets host EKS worker nodes, RDS, and ElastiCache. Route53 handles DNS with CloudFront as CDN for static assets on S3.

■ Compute & Orchestration:

EKS managed node groups (t3.medium) run Kubernetes workloads. Each microservice has its own Deployment, ClusterIP Service, and HPA. Kustomize overlays manage environment-specific configs across dev, staging, and prod.

■ CI/CD Flow:

Jenkins pipelines trigger on branch commits — feature branches deploy to dev, main deploys to staging, and release tags promote to prod. Docker images are stored in ECR with environment-tagged versions.

■ Data & Observability:

RDS PostgreSQL (Multi-AZ for prod) handles persistent data. Datadog APM traces service calls; Prometheus scrapes metrics; Grafana dashboards visualize SLIs. Alerts fire to Slack via Datadog monitors.

Q4. What DevOps tools are you currently using and why?

■ Terraform:

Infrastructure as Code for AWS provisioning. Chosen for its declarative syntax, strong AWS provider support, state management, and modularity. Enables consistent, reproducible environments.

■ Jenkins:

CI/CD orchestration. Used for its flexibility in pipeline scripting, broad plugin ecosystem, and ability to run dynamic agent-based builds on Kubernetes pods via the Kubernetes plugin.

■ Docker & ECR:

Containerization for consistent build artifacts. Multi-stage Dockerfiles minimize image size. ECR integrates natively with EKS using IAM roles — no external registry credentials needed.

■ Kubernetes / EKS:

Container orchestration for scaling, self-healing, and rolling deployments. EKS offloads control plane management to AWS, reducing operational overhead.

■ Kustomize:

Manages environment-specific Kubernetes manifests without templating complexity. Overlays for dev/staging/prod reduce duplication and config drift.

■ Datadog + Prometheus/Grafana:

Datadog for APM, distributed tracing, and synthetic monitoring. Prometheus/Grafana for infrastructure metrics. Combined approach provides both application-layer and infra-layer observability.

■ SonarQube + Trivy:

SonarQube enforces code quality gates (coverage, code smells, vulnerabilities). Trivy scans container images for CVEs before ECR push — security integrated into the pipeline.

Q5. What were the biggest challenges you faced in your current project?

■ Secrets Management:

Initially stored credentials as base64-encoded Kubernetes Secrets — which is not encryption. The gap was recognized and the path forward involves External Secrets Operator pulling secrets from AWS Secrets Manager at runtime, eliminating plaintext secrets from manifests.

■ Pipeline Slowness & Optimization:

Jenkins pipelines initially ran sequentially. Restructured independent stages (test, scan, build) to run in parallel, added Docker layer caching, and configured Maven dependency caching via PVC mounts on Jenkins agents — reducing build times significantly.

■ EKS IAM Complexity:

Mapping pod-level AWS permissions without hardcoding access keys required implementing IRSA (IAM Roles for Service Accounts) via OIDC federation — a non-trivial setup that required understanding the trust policy relationship between Kubernetes service accounts and IAM.

■ Multi-Environment Config Drift:

Managing config differences across dev/staging/prod without duplication was solved by Kustomize overlays — base manifests define the common spec, overlays apply environment-specific patches like replica counts and resource limits.

■ Observability Gaps:

Early monitoring covered only infra metrics. Added Datadog APM instrumentation to microservices to capture distributed traces, enabling root cause identification for latency spikes across service boundaries.

02 SECTION 2: CI/CD — Jenkins

Q6. Explain your Jenkins pipeline architecture.

■ Pipeline Type:

Declarative pipeline defined in a Jenkinsfile committed to the application repository — pipeline-as-code, versioned alongside source.

■ Agent Strategy:

Master node runs orchestration only. Builds execute on dynamic Kubernetes pod agents provisioned via the Kubernetes plugin. Each build gets a fresh pod with containers for Maven, Docker, and kubectl.

■ Stage Structure:

Stages run in order: Checkout → Build (Maven) → Unit Tests → SonarQube Quality Gate → Docker Build → Trivy Scan → ECR Push → Kustomize Deploy → Health Check. Independent stages like tests and scans are parallelized.

■ Branch Strategy:

Feature branches deploy to dev automatically. Merges to main trigger staging deploy with an approval gate. Release tags trigger prod deployment with mandatory sign-off via Jenkins input step.

■ Notifications:

Post-build steps send Slack notifications with stage status, commit author, and build duration on both success and failure.

Q7. Difference between Declarative and Scripted Pipeline.

■ Declarative Pipeline:

Structured syntax using predefined blocks — pipeline {}, agent {}, stages {}, steps {}. Enforces a strict schema, making it easier to read, maintain, and validate. Supports built-in directives like post {}, when {}, and options {}. Recommended for most use cases.

■ Scripted Pipeline:

Groovy-based, free-form code in a node {} block. No imposed structure — full programmatic control using loops, conditionals, try/catch. More powerful but harder to read and maintain at scale.

■ When to Use Each:

Declarative for standard pipelines where readability and convention matter. Scripted when you need complex logic, dynamic stage generation, or advanced Groovy constructs not expressible in declarative syntax.

■ Key Practical Difference:

In Declarative, syntax errors are caught at pipeline parse time — fast feedback. In Scripted, errors surface at runtime, which is harder to debug in complex pipelines.

Q8. How do you handle pipeline failures?

■ Post Block Handling:

Declarative pipelines use the post {} block with conditions: always (cleanup), success (notify green), failure (notify red + alert), unstable (test failures without build errors), and aborted (manual stops).

■ Try/Catch in Scripted:

Scripted pipelines wrap critical sections in try/catch/finally blocks. Caught exceptions trigger rollback steps or notifications before re-throwing.

■ Retry Mechanism:

Transient failures (network timeouts, flaky tests) are wrapped with the retry(n) step to attempt up to n times before marking as failed.

■ Timeout Guards:

timeout(time: 10, unit: 'MINUTES') prevents zombie builds from hanging indefinitely — especially important for Docker builds and deployment health checks.

■ Rollback on Deploy Failure:

If the deployment health check stage fails, a rollback stage executes kubectl rollout undo to revert to the previous ReplicaSet. The pipeline marks the build as failed and pages on-call via Datadog.

Q9. What stages do you typically include in a CI/CD pipeline?

■ Stage 1 — Checkout:

Clone the repository from GitHub with the specific branch/tag that triggered the build.

■ Stage 2 — Build:

Compile source code. For Java: `mvn clean package -DskipTests`. Build artifacts are versioned with the build number and commit SHA.

■ Stage 3 — Unit Tests:

Run unit tests and generate coverage reports. Published to Jenkins via JUnit plugin. Pipeline fails if coverage drops below threshold.

■ Stage 4 — Static Analysis (SonarQube):

Run SonarQube scanner. Pipeline pauses at quality gate — fails if critical issues or coverage thresholds are not met.

■ Stage 5 — Docker Build:

Build container image using multi-stage Dockerfile. Image tagged with: `servicename:gitcommitsha-buildnumber`.

■ Stage 6 — Security Scan (Trivy):

Scan Docker image for CVEs. Pipeline fails on HIGH or CRITICAL severity vulnerabilities. Results published as a build artifact.

■ Stage 7 — Push to ECR:

Authenticate to ECR using AWS IAM role (via Jenkins IAM instance profile). Push tagged image.

■ Stage 8 — Deploy:

Update Kustomize image tag in the overlay, run `kubectl apply`. For prod: manual approval gate before deploy.

■ Stage 9 — Health Check:

Poll `kubectl rollout` status for up to 5 minutes. If rollout does not complete, trigger rollback and fail the build.

Q10. How do you secure Jenkins credentials?

■ Jenkins Credentials Store:

All secrets stored in Jenkins Credentials Manager — never hardcoded in Jenkinsfiles. Credentials accessed via the `credentials()` binding or with `Credentials {}` block and injected as environment variables at runtime.

■ AWS IAM Role (No Static Keys):

Jenkins master EC2 instance runs with an IAM instance profile. No AWS access key or secret key stored anywhere — the SDK uses the instance metadata service for temporary credentials.

■ Docker Registry Credentials:

ECR login handled via `aws ecr get-login-password` piped to `docker login` — short-lived token, never a stored password.

■ Credential Masking:

Jenkins automatically masks credential values in build logs. Avoid `echo $SECRET` in shell steps — masked values can be exposed via indirect variable expansion.

■ Folder-Scoped Credentials:

Credentials scoped to Jenkins Folders rather than global scope. Service-specific credentials are only accessible to pipelines within their designated folder.

■ Rotation Policy:

Recommend rotating all stored credentials quarterly and using AWS Secrets Manager + External Secrets Operator for application-level secrets rather than injecting them through Jenkins.

Q11. Have you configured Jenkins agents dynamically?

■ Kubernetes Plugin:

Configured the Kubernetes plugin on Jenkins master to provision build pods on-demand in the EKS cluster. Each pipeline defines a podTemplate {} block specifying containers (jnlp, maven, docker) and resource requests/limits.

■ Pod Template Definition:

The JNLP container connects the agent back to the Jenkins master. Additional containers (maven:3.8, docker:dind) are sidecar containers that execute specific build steps using the container() {} block in the pipeline.

■ Lifecycle:

Pods spin up at pipeline start, execute all stages, then terminate automatically. No persistent agents to maintain — eliminates agent drift and stale environments.

■ Resource Management:

Resource requests defined per container: e.g., maven gets 1 CPU / 2Gi memory for builds. Kubernetes schedules pods on appropriate nodes respecting cluster capacity.

■ Benefit:

Build isolation — each run starts from a clean pod image. Horizontal scalability — multiple builds run concurrently without agent contention. No manual agent provisioning.

Q12. How do you optimize a slow Jenkins pipeline?

■ Identify the Bottleneck First:

Check the Pipeline Stage View or Blue Ocean timing breakdown. Identify which stage consumes the most time before making changes — avoid optimizing the wrong stage.

■ Parallelize Independent Stages:

Stages with no dependency on each other (e.g., unit tests + SonarQube scan + Trivy scan) run concurrently using the parallel {} block. This alone can cut total time by 30-50%.

■ Cache Maven Dependencies:

Mount a PersistentVolumeClaim at the .m2/repository path on Jenkins agent pods. Avoids re-downloading all dependencies on every build — high-impact optimization for Java projects.

■ Docker Layer Caching:

Order Dockerfile instructions from least-to-most frequently changing. COPY pom.xml + RUN mvn dependency:go-offline before COPY src/. This caches dependency download layers across builds.

■ Shallow Git Clone:

Add --depth=1 to git clone. Avoids fetching full history. For large repos, this saves tens of seconds.

■ Skip Redundant Stages:

Use when {} conditions to skip stages that don't apply — e.g., skip deploy stage for documentation-only commits by checking changed file paths.

■ Tune Agent Resources:

Underpowered build agents (low CPU, memory pressure) cause slow Maven compiles and Docker builds. Right-size pod resource requests based on observed usage.

Q13. Explain rollback strategy in Jenkins deployment.

■ **Kubernetes Native Rollback:**

Kubernetes maintains a rollout history for Deployments. If the health check stage fails post-deploy, the pipeline executes: `kubectl rollout undo deployment/ -n` . This reverts to the previous ReplicaSet immediately.

■ **Image Tag Pinning:**

Every deploy uses a specific image tag (commit SHA). Rollback means re-deploying the last known good tag — either manually or via re-triggering the previous successful pipeline build.

■ **Kustomize Rollback:**

In the GitOps approach, rolling back means reverting the image tag in the Kustomize overlay to the previous commit. This is done with `git revert` + pipeline re-run — fully auditable.

■ **Blue-Green Rollback:**

With blue-green deployments, rollback is a DNS/ALB weighted routing switch — 100% traffic moved back to the stable (blue) environment. Zero downtime, instant recovery.

■ **Automated vs Manual:**

For automated rollbacks: health check failure triggers `kubectl rollout undo` automatically. For prod: a rollback requires explicit re-approval from the team lead to maintain change control.

■ **Post-Rollback Action:**

After rollback, the pipeline sends a critical alert to Slack and creates an incident in the tracking system. The failed build is marked as unstable, not deleted, for post-mortem analysis.

■ SCENARIO: Jenkins pipeline: 15 min → 45 min. How do you troubleshoot?

■ STEP 1 — CHECK STAGE TIMINGS: Open Blue Ocean or Pipeline Stage View. Identify which specific stage(s) grew. Determine if the slowdown is in one stage (e.g., Maven build went from 4 min to 20 min) or spread across all stages (suggesting an infrastructure issue).

■ STEP 2 — CHECK RECENT CHANGES: Compare the first slow build against the last fast build. Was a new dependency added (pom.xml change)? Was a new pipeline stage introduced? Was the Docker base image changed? Correlation with a code or config change narrows root cause immediately.

■ STEP 3 — MAVEN / DEPENDENCY DOWNLOAD: If the Maven build stage is slow, check whether the .m2 cache was lost. A PVC mount may have been deleted, or the pod was scheduled on a new node without a warm cache — forcing re-download of all dependencies. Fix: restore PVC or re-warm cache.

■ STEP 4 — DOCKER BUILD LAYER INVALIDATION: If Docker build stage is slow, layer cache may have been busted. Check if a frequently-changing file (like application.properties) was moved earlier in the Dockerfile, invalidating all downstream layers. Fix: reorder Dockerfile layers — static dependencies before source code.

■ STEP 5 — SONARQUBE / TRIVY SCAN SLOWDOWN: Check if SonarQube server response time degraded (disk I/O, memory pressure on Sonar host). Check if Trivy is re-downloading its vulnerability DB on every build instead of caching it. Fix: mount Trivy DB cache volume on agent pods.

■ STEP 6 — AGENT RESOURCE PRESSURE: Check if Jenkins Kubernetes agents are running on a node with high CPU/memory utilization. Low resource availability causes build steps to run slower. `kubectl top nodes` and `kubectl describe node` will reveal pressure. Fix: adjust pod resource requests or scale up the node group.

■ STEP 7 — NETWORK LATENCY: If the pipeline interacts with external services (ECR, SonarQube, GitHub), check network latency or rate limits. A saturated NAT Gateway or VPC bandwidth constraint can slow all external calls. Check CloudWatch NAT Gateway metrics.

■ STEP 8 — PARALLEL STAGES REMOVED: Check if a recent Jenkinsfile change inadvertently serialized previously parallel stages. A well-meaning refactor might have moved parallel `{}` blocks into sequential execution. Restore parallelism for independent stages.

■ STEP 9 — REMEDIATE & PREVENT: After fixing, add pipeline timing assertions — if total build time exceeds 20 minutes, generate a warning notification. Track build duration as a metric in Datadog to catch regression early.

Interview Tip: Always anchor answers in your ApnaKart project experience. Use the pattern: *situation* → *action* → *result*. When discussing gaps (e.g., ESO, Vault), acknowledge them and describe your learning path — interviewers respect self-awareness over overstatement.